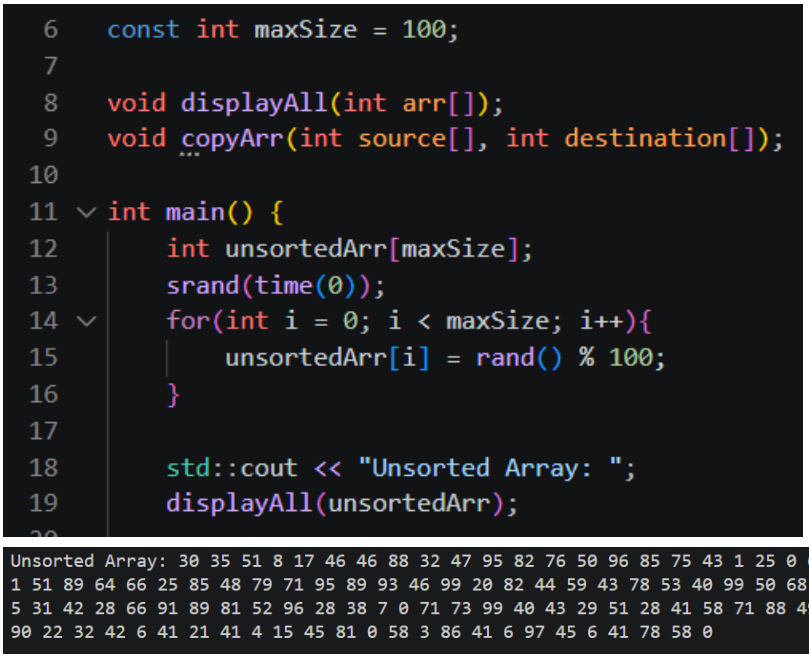
COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 | Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	Sep 15, 2026	**DATE SUBMITTED:**	Sep 15, 2026
NAME:	Serrano, Gabriel Immanuel	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 8.1 Sorting Algorithms Pt2

1. PROCEDURE OUTPUT

Code + Console Screenshot	
Observations	The unsortedArray contains 100 randomly generated values that range from 0 to 99 using rand() % 100, which limits the generated values to 99. Then srand(time(0)) makes the values different each time and the for loop is used to fill all indexes. We also defined a displayAll and copyArr to display the array and copy the values of the unsorted array.

```
 6   const int maxSize = 100;
 7
 8   void displayAll(int arr[]);
 9   void copyArr(int source[], int destination[]);
10
11   int main() {
12       int unsortedArr[maxSize];
13       srand(time(0));
14       for(int i = 0; i < maxSize; i++){
15           unsortedArr[i] = rand() % 100;
16       }
17
18       std::cout << "Unsorted Array: ";
19       displayAll(unsortedArr);
```

```
Unsorted Array: 30 35 51 8 17 46 46 88 32 47 95 82 76 50 96 85 75 43 1 25 0 6
1 51 89 64 66 25 85 48 79 71 95 89 93 46 99 20 82 44 59 43 78 53 40 99 50 68 5
5 31 42 28 66 91 89 81 52 96 28 38 7 0 71 73 99 40 43 29 51 28 41 58 71 88 49
90 22 32 42 6 41 21 41 4 15 45 81 0 58 3 86 41 6 97 45 6 41 78 58 0
```

Table 8-1. Array of Values for Sort Algorithm Testing

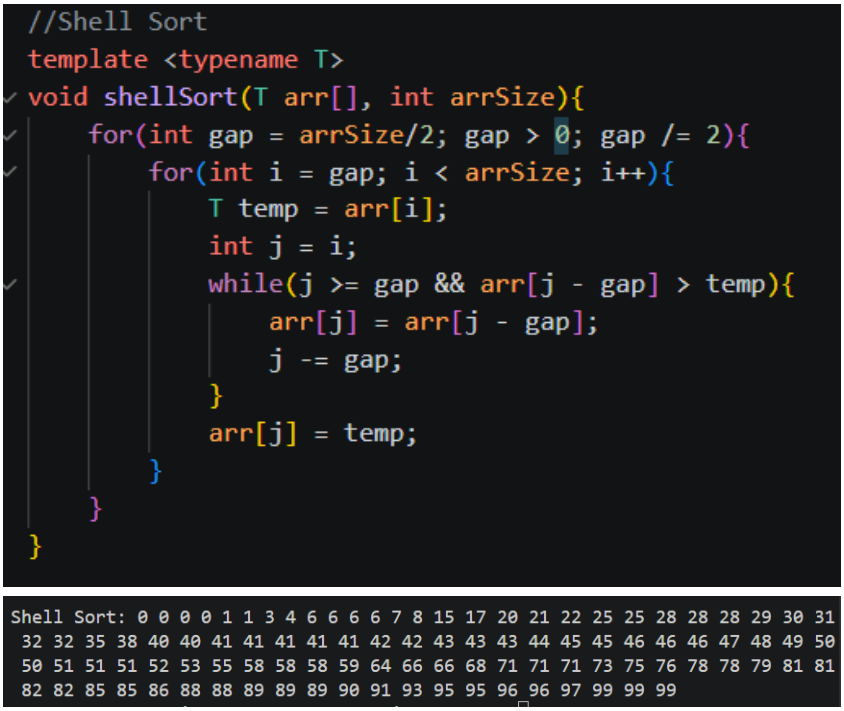
Code + Console Screenshot	
Observations	In the shellSort function, we used two for loops and a while loop to compare the elements of the array and arrange them in ascending order. It works by using a gap to create smaller sublists, where the elements are compared based on their distance from each other. The while loop moves the larger element forward when the array is greater than the temp, then places the temp in its proper position until the gap decreases into 1, making the element to be arranged in sorted order.

Table 8-2. Shell Sort Technique

Code + Console Screenshot

```cpp
//merge sort helper
template <typename T>
void merge(T arr[], int left, int mid, int right){
    int n1 = mid - left + 1;
    int n2 = right - mid;

    T* leftArr = new T[n1];
    T* rightArr = new T[n2];

    for(int i = 0; i < n1; i++){
        leftArr[i] = arr[left + i];
    }
    for(int j = 0; j < n2; j++){
        rightArr[j] = arr[mid + 1 + j];
    }

    int i = 0 , j = 0, k = left;
    while(i < n1 && j < n2){
        if(leftArr[i] <= rightArr[j]){
            arr[k] = leftArr[i];
            i++;
        }
        else{
            arr[k] = rightArr[j];
            j++;
        }
        k++;
    }
```

```cpp
    while(i < n1){
        arr[k] = leftArr[i];
        i++;
        k++;
    }
    while(j < n2){
        arr[k] = rightArr[j];
        j++;
        k++;
    }
}

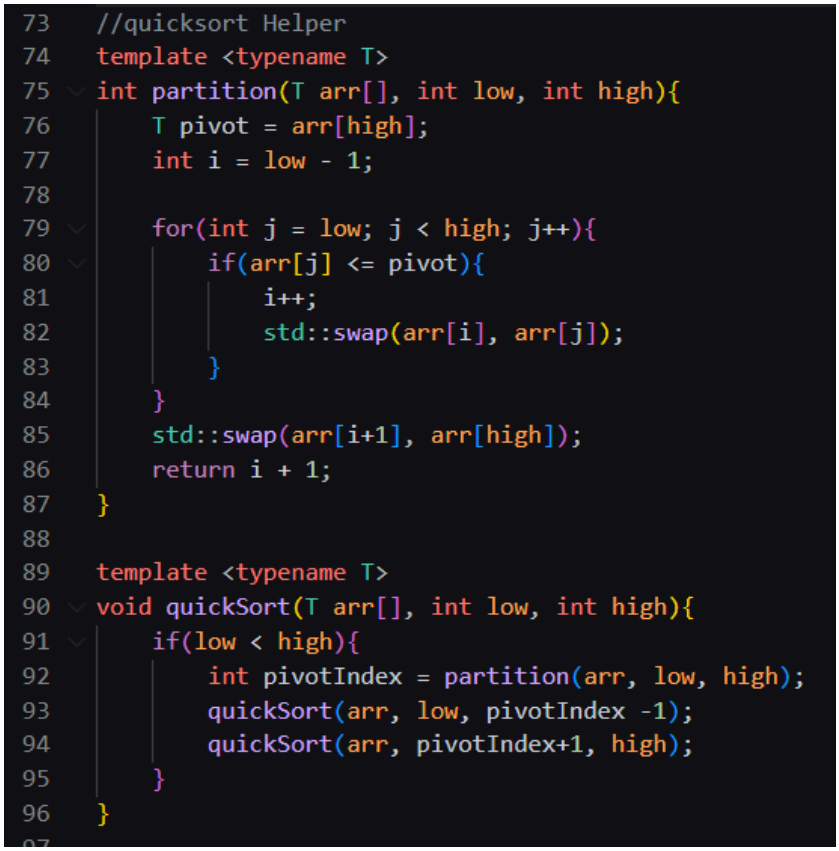
//merge sort
template <typename T>
void mergeSort(T arr[], int left, int right){
    if(left < right){
        int mid = (left + right) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
```

```
Merge Sort: 0 0 1 2 2 3 4 4 9 10 11 11 13 14 15 18 22 24 24 25 27 27 27 28 29 29
29 31 34 35 35 36 36 37 42 42 42 42 44 44 44 45 45 47 48 51 52 53 53 54 54 56 56
57 57 57 59 60 60 61 62 62 65 66 68 68 71 73 75 75 76 77 78 79 81 82 82 82 83 83
85 85 87 89 90 90 91 92 93 93 93 94 95 96 96 96 97 98 98 99
```

Observations	In the mergeSort function, we used a helper function to divide the array into smaller lists until each list has only one element. Then it creates two temp arrays for the left and right and compares their elements to arrange them in ascending order. If the element in the left sublist is smaller, it is placed into the original array first, otherwise the element from the right sublist is placed first. The remaining elements are then copied into the original array, and this process continues until the whole array is sorted.

Table 8-3. Merge Sort Algorithm

Code + Console Screenshot	

```
73  //quicksort Helper
74  template <typename T>
75  int partition(T arr[], int low, int high){
76      T pivot = arr[high];
77      int i = low - 1;
78
79      for(int j = low; j < high; j++){
80          if(arr[j] <= pivot){
81              i++;
82              std::swap(arr[i], arr[j]);
83          }
84      }
85      std::swap(arr[i+1], arr[high]);
86      return i + 1;
87  }
88
89  template <typename T>
90  void quickSort(T arr[], int low, int high){
91      if(low < high){
92          int pivotIndex = partition(arr, low, high);
93          quickSort(arr, low, pivotIndex -1);
94          quickSort(arr, pivotIndex+1, high);
95      }
96  }
```

```
Quick Sort: 0 1 1 3 4 5 5 6 6 6 6 7 7 12 14 16 16 16 16 17 18 19 19 24 25 25 26
 27 28 29 29 30 31 32 33 33 33 35 36 39 39 42 42 42 43 43 45 47 48 51 51 53 53
53 54 54 54 55 57 57 58 58 60 61 61 62 65 65 66 66 71 71 71 73 74 76 76 76 77 7
7 78 78 79 80 83 84 85 85 86 87 89 89 90 90 90 94 94 95 97 97
```

Observations	In the quickSort function, we used a helper function to divide the array into smaller sublists. The partition function uses the last element as the pivot and checks each element to see if it is less than or equal to the pivot, then it swaps it into the correct position in the index. After the pivot is placed in the proper position, the quickSort function is called again for the left and right sublists until the array is sorted. The time complexity of this is O(n log n) because the array is divided into smaller parts each time it runs the loop.

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

Yes, once using the partition method in Quick Sort, it divides the array into a left sublist and a right sublist, and we can use other sorting algorithms to sort these sublists.

```cpp
#ifndef SUPP_H
#define SUPP_H

template <typename T>
void InsertionSort(T arr[], int arrSize){
    int k = 1, j;
    T temp;

    while(k < arrSize){
        temp = arr[k];
        j = k -1;
        while(j >= 0 && temp <= arr[j]){
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = temp;
        k++;
    }
}

//quicksort Helper
template <typename T>
int partition(T arr[], int low, int high){
    T pivot = arr[high];
    int i = low - 1;

    for(int j = low; j < high; j++){
        if(arr[j] <= pivot){
            i++;
            std::swap(arr[i], arr[j]);
        }
    }
    std::swap(arr[i+1], arr[high]);
    return i + 1;
}

template <typename T>
void quickSortInsert(T arr[], int low, int high){
    if(low < high){
        int pivotIndex = partition(arr, low, high);
        InsertionSort(arr + low, pivotIndex - low);
        InsertionSort(arr + pivotIndex + 1 , high-pivotIndex);
    }
}
#endif
```

```cpp
#include <iostream>
#include "supp.h"

void displayAll(int arr[], int arrSize);

int main(){
    int exampleArr[] = {42, 21, 10, 15, 57, 67};
    int exampleSize = 6;

    std::cout << "Before sorting: ";
    displayAll(exampleArr, exampleSize);

    quickSortInsert(exampleArr, 0, exampleSize - 1);

    std::cout << "After sorting: ";
    displayAll(exampleArr, exampleSize);

    return 0;
}

void displayAll(int arr[], int arrSize){
    for(int i = 0; i < arrSize; i++){
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}
```

```
Before sorting: 42 21 10 15 57 67
After sorting: 10 15 21 42 57 67
```

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

- For the given array, I think Merge Sort can give the best time performance because Merge Sort repeatedly divides the array into smaller lists and then merges them back in sorted order. Merge Sort and Quick Sort can have O(N • log N) for their time complexity because they both use a divide and conquer approach. They usually divide the array into smaller parts, each time the loop is called, the number of divisions increases while the number of elements in each part becomes smaller, giving us the time complexity of O(N • log N)

Conclusion :
I learned how to implement different sorting algorithms and how these algorithms can be used to organize values in an array. Through this activity, I learned the proper usage of Shell Sort, Merge Sort, and Quick Sort. For example, the Shell Sort sorts elements by using different gaps to create smaller lists, Merge Sort divides the array into smaller lists and merges them back in sorted order, while the Quick Sort uses a pivot to partition the array into a left and right list. For the supplementary activity it is about understanding the concepts of a sorting algorithm because of this I was able to create a hybrid of quick sort and insertion sort. Overall, I was able to understand and implement the sorting algorithms correctly. This activity also made me understand how the algorithm divides and creates sublists to sort an array into sorted order.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT and Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar Correction and Code Assistance and Explanation of Concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Grammarly was used for grammar correction, and ChatGPT was used for assisting and debugging the code and it was also used to explain the concepts of how the approach should I take in creating the algorithm that was in Pseudocode.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.


T.I.P
TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY